FlexQL — Design Document

GitHub Repository

https://github.com/naninikhil21/flexQL/tree/submission_branch

---

1. How the Data Is Stored

1.1 In-Memory Row Store

FlexQL stores all table data in a custom **RowStore** — a contiguous, append-only byte buffer where every row is a fixed-size binary record. Field layouts are computed once at `CREATE TABLE` time and stored in a `Schema` object alongside each table.

Column type encodings and sizes:

SQL Type	C++ Representation	Size in Row Buffer
`DECIMAL` / `INT`	`int64_t`	8 bytes
`FLOAT` / `REAL`	`double`	8 bytes
`DATETIME`	`int64_t` (epoch)	8 bytes
`VARCHAR(N)` / `TEXT`	`char[N+1]`	N + 1 bytes (null-term.)

Each column stores its byte **offset** inside the schema, computed as a running sum of preceding column sizes. A row in memory is simply a `char[]` of size `schema.row_size`. Reads and writes use `memcpy` directly into/from the appropriate slot.

The `RowStore` also holds a per-table mutex (`_row_mu`) so that concurrent inserts from multiple client threads are serialised safely at the row-append level.

1.2 Write-Ahead Log (WAL) — Durable Persistence

All mutations are durably written to `flexql.wal` before being acknowledged, providing crash-recovery guarantees.

Record format (v4):

```
CREATE TABLE record:
  [4B magic 0xF1E4D801] [4B payload_length]
  [4B table_name_len]   [table_name bytes]
  [4B col_count]
  { [4B col_name_len] [col_name] [1B type_tag] [4B varchar_len] } × N
  [4B pk_col_index  (0xFFFFFFFF = no primary key)]

INSERT BATCH record:
  [4B magic 0xF1E4D802] [4B table_name_len] [table_name]
  [8B row_count] [8B row_size]
  [raw row bytes: row_count × row_size]

DROP TABLE record:
  [4B magic 0xF1E4D803] [4B table_name_len] [table_name]
```

The `payload_length` field added in v4 allows forward-compatible parsing — future fields can be appended and old readers will skip them safely.

WAL lifecycle: 1. `replay()` on startup reads the WAL sequentially, reconstructing in-memory schemas and rows. 2. `append_create_table()` / `append_batch()` / `append_drop_table()` queue records for the background writer. 3. `checkpoint_from_catalog()` truncates the WAL and rewrites it from the current in-memory state, controlling unbounded file growth. Checkpointing runs on startup (after replay) and on shutdown.

---

2. Indexing Method

2.1 Primary Key Hash Index

Tables declared with `PRIMARY KEY` on any column get a per-table **Robin Hood open-addressing hash index** (`HashIndex<int64_t, uint32_t>`).

Design details:

- **Hash function:** 64-bit multiplicative hash (`key * 11400714819323198485ULL >> 32`) — a Knuth/Fibonacci hash variant that distributes integer keys uniformly.
- **Collision resolution:** Robin Hood probing — during insertion, an incoming key steals the slot from a "richer" key (one closer to its home slot) and the displaced key continues probing. This minimises the maximum probe distance.
- **Load factor threshold:** 90%. When exceeded, the table doubles capacity and rehashes all entries. The probe `dib` (distance-from-bucket) counter starts at 1 on both insert and lookup to maintain consistency.
- **Key type:** `int64_t` (the in-memory encoding for DECIMAL, INT, DATETIME PK columns).
- **Purpose:** Duplicate key detection on insert; O(1) average-case unique constraint enforcement.

The `pk_col_index` field (stored in `Schema` and persisted in WAL) identifies which column is the primary key, supporting PK on any column position — not just the first.

2.2 Sequential Scan (Non-PK Tables)

Tables without a primary key use a full sequential scan for `SELECT` and `WHERE` evaluation. The scan iterates over the flat row buffer in cache-friendly linear order, checking each row against the `WHERE` chain.

---

3. Caching Strategy

3.1 LRU Query Result Cache

The server maintains an **LRU (Least Recently Used) query result cache** keyed on normalized SQL text (lowercased, trimmed).

- **Structure:** An ordered map of `sql_text → result_string` pairs. When capacity is exceeded, the least-recently-used entry is evicted.
- **Size:** Configurable via the `FLEXQL_CACHE_SIZE` environment variable (default: 64 entries).
- **Cache hits:** The result is returned immediately without executing the query, providing sub-microsecond response for repeated reads.

3.2 Cache Invalidation

Cache entries are aggressively invalidated on any write: - `INSERT INTO <table>` evicts all cache entries whose SQL references `<table>`. - `DROP TABLE <table>` evicts all entries referencing `<table>`. - `CREATE TABLE` is a schema change and triggers invalidation of any cached queries referencing that table name.

`SELECT COUNT(*)` queries are **not cached** — aggregate results would immediately become stale after any insert.

---

4. Handling of Expiration Timestamps

FlexQL supports automatic row expiration for tables that include an **`expires_at`** column (`DECIMAL`, `INT`, or `DATETIME` type):

- The column stores a Unix epoch timestamp (seconds since 1970-01-01 UTC) as an `int64_t`.
- During `SELECT` scans, each row is checked: if `expires_at < now_epoch_seconds()`, the row is **silently skipped** and excluded from results.

- No background sweeper thread exists — expiration is evaluated **lazily at query time**. This avoids global lock contention and keeps the hot-path simple.
- Rows are never physically deleted: they remain in the WAL and in memory, but are simply not visible to queries after expiry. A checkpoint compacts them away implicitly.

---

5. Multithreading Design

5.1 Thread-per-Client Model

The server spawns one OS thread per accepted TCP connection. Each thread independently: 1. Reads SQL from the socket into a buffer. 2. Sends the SQL through the parser and executor. 3. Writes results back to the socket.

Client threads share access to the Catalog (schema store) and all `RowStore` objects.

5.2 Synchronisation Primitives

Resource	Mechanism	Scope
Catalog (schema map)	`std::shared_mutex` (reader–writer)	Multiple readers; exclusive writer
Per-table rows (`RowStore`)	`std::mutex` (`_row_mu`)	Row-level append serialisation
WAL pending queue	`std::mutex` + `std::condition_variable`	Producer (client threads) / consumer (WAL writer thread)
Query result cache	`std::mutex`	Protect LRU map reads and eviction

5.3 Background WAL Writer Thread

A single dedicated writer thread drains the pending WAL record queue using `writev()` for scatter-gather I/O, batching multiple records into one kernel call for throughput. Client threads never wait for disk I/O to complete — they queue a record and return immediately.

5.4 Design Rationale

Thread-per-client was chosen over event-driven I/O because: - The SQL protocol is synchronous and request–response — one statement at a time per connection — making event multiplexing unnecessary. - Thread-per-client gives simple, correct read-after-write ordering per connection for free (via sequential execution on the same thread). - The largest gains come from decoupling disk I/O into the background writer, not from replacing the threading model.

---

6. Other Design Decisions

SQL Parsing — Recursive Descent

A hand-written recursive descent parser operates directly on `string_view` tokens from a zero-copy lexer. This avoids heap allocations in the parse path, remains fully predictable and debuggable, and is sufficient for the supported SQL subset. The parser enforces end-of-input after every statement so unsupported clauses (`ORDER BY` , `GROUP BY` , etc.) are correctly rejected with a clear error.

Table Alias Support

`FROM table AS alias` and `JOIN table AS alias` are supported. The parser distinguishes the `AS` keyword (a KEYWORD token) from implicit aliases (bare IDENTIFIER tokens) using `accept_kw("AS")` before falling back to the identifier branch.

Strict WHERE Column Aliasing

`eval_where` validates that any `table_alias` prefix on a WHERE column matches the schema's table name, preventing silent column mismatches in multi-table queries.

No SQLite Dependency

The entire engine — storage, indexing, WAL, parsing, execution — is custom C++17 with no third-party database library. The only external dependency is `liburing` for efficient I/O on Linux.